

Lab - Device Communication with MQTT and Python

Introduction to MQTT

- **Message Queueing Telemetry Transport (MQTT)** is a lightweight, publish-subscribe modelled protocol that runs over TCP/IP stack, widely used in IoT networking.

Agenda for this hands on :

- Install an MQTT broker on your local computer
- Interact with the MQTT broker using CLI
- Configure the listener
- Communicate with the local MQTT broker using a Python program

Duration:

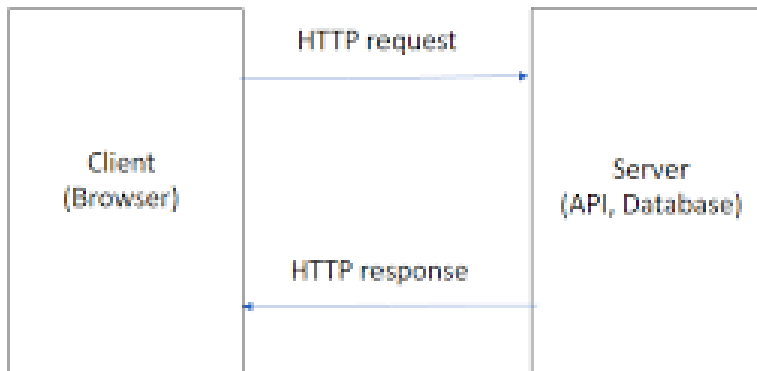
- This lab requires approximately **2 hours** to complete.

Keynote:

- It's important that you complete all tasks in this hands on as it will lay foundation for you upcoming session.

Communication models

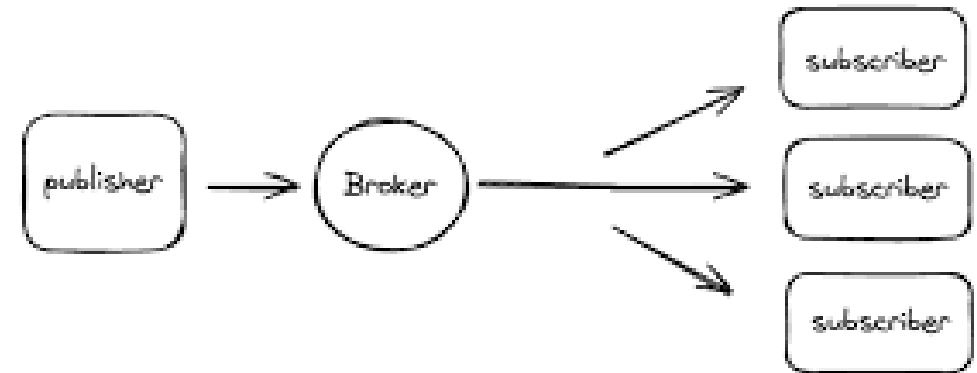
Request Response (Client/Server) communication model



Popular Request / response protocols in IoT

- HTTP/REST
- gRPC
- SOAP

Publisher Subscriber (Pub-Sub) communication model



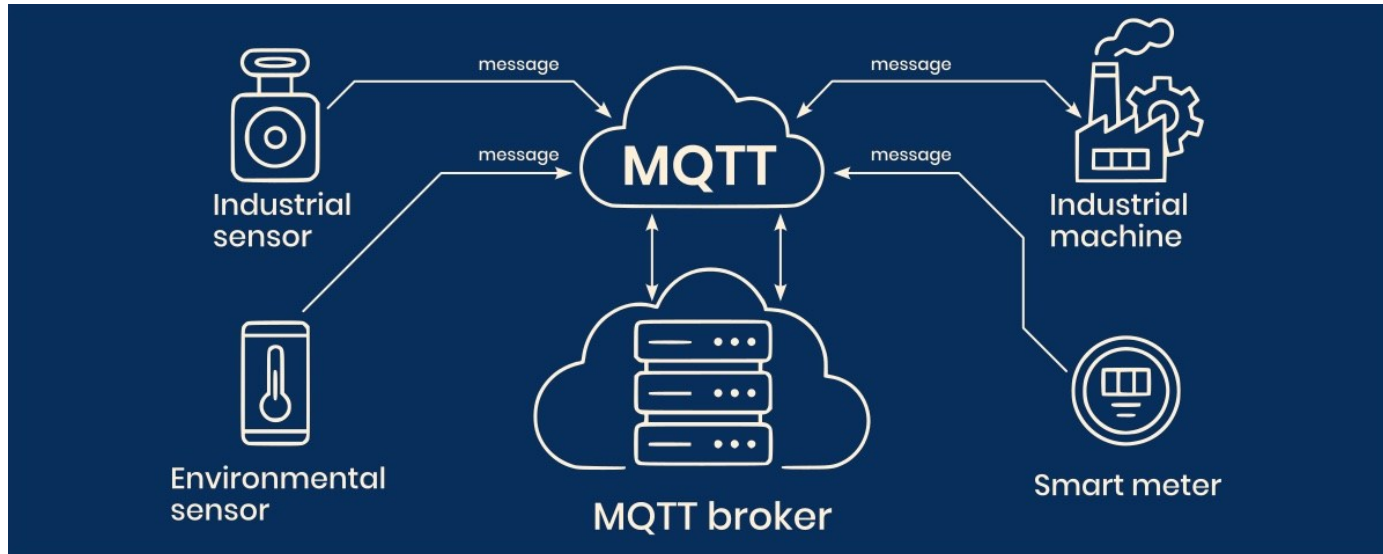
Popular Request / response protocols in IoT

- MQTT
- AMQP
- NATS

What is MQTT ?

MQTT - Message Queuing Telemetry Transport

- Lightweight pub/sub messaging protocol. Designed for IoT & M2M communication.
- Widely used in
 - **IoT & IIoT:** Connecting sensors & embedded devices.
 - **Home Automation:** Interconnecting smart systems.
 - **Remote Monitoring:** Oil, gas, and utility monitoring



Key Components

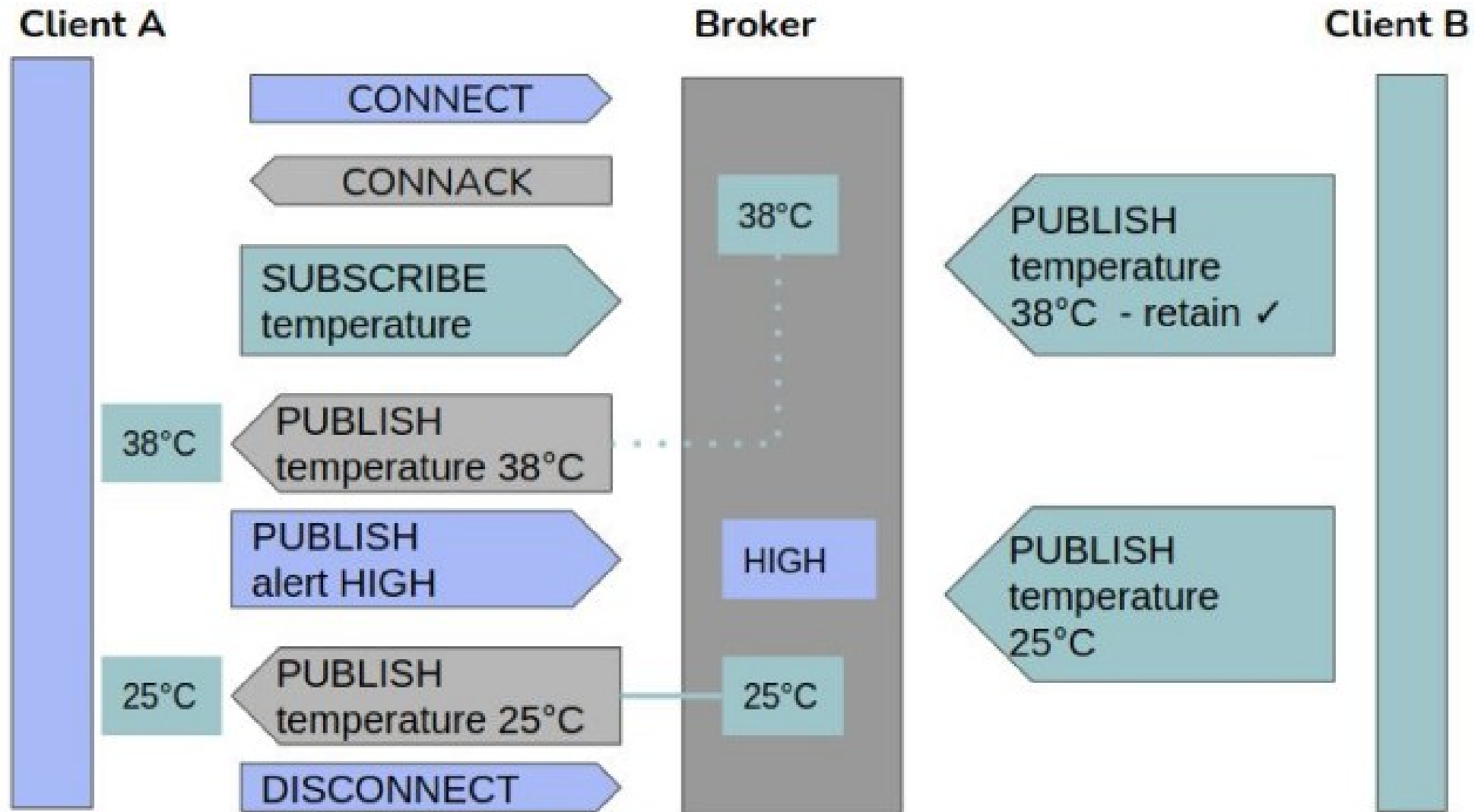
Broker: Acts as the central hub, receiving messages from publishers and distributing them to subscribers.

Publisher: Devices/sensors that send data to a specific topic.

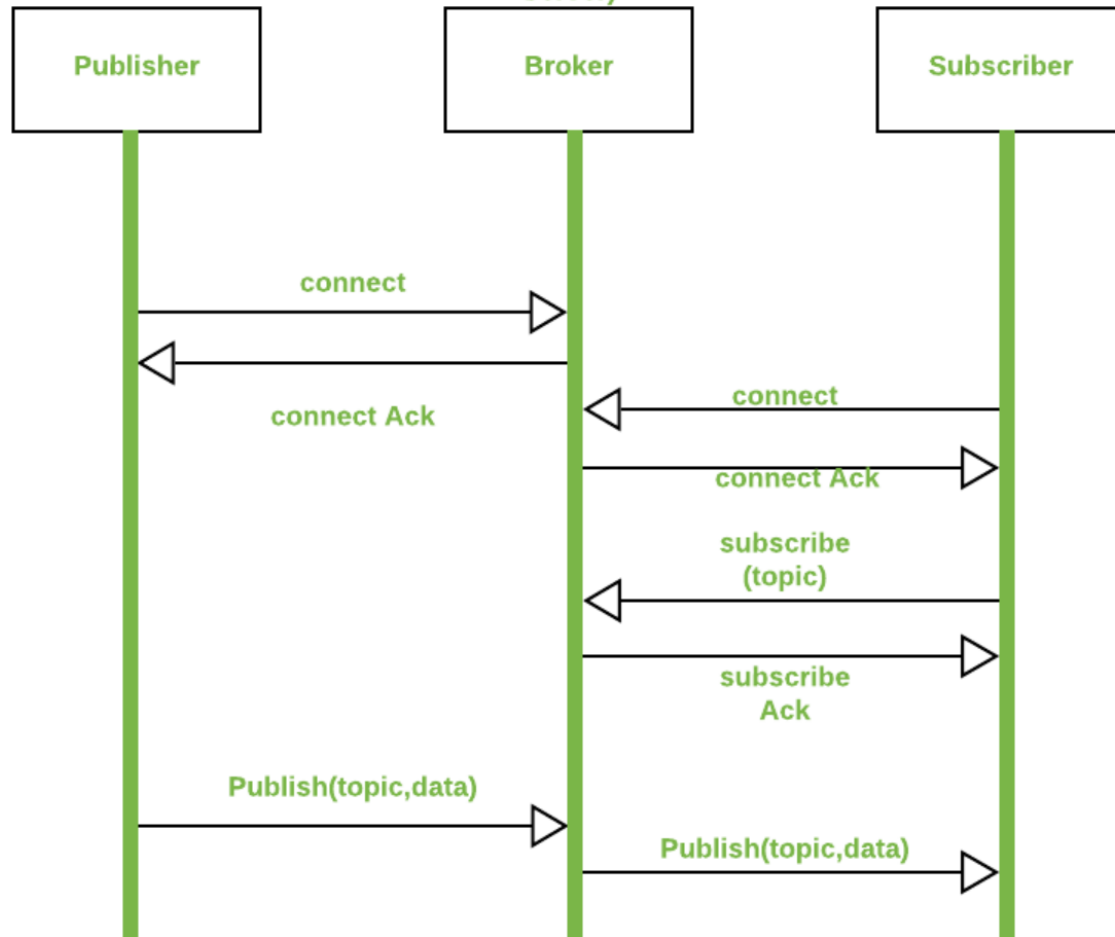
Subscriber: Devices that express interest (subscribe) to specific topics to receive data.

Topic: The mechanism for routing messages.

How MQTT Works ?



****Task 1** Download and Start the broker**



Popular MQTT Broker

- HiveMQ
- Mosquitto
- RabbitMQ
- MQTT Route
- Learn more on -<https://mqtt.org/software/>

Mosquito broker

- <https://www.mosquitto.org/>

Lab 2 - Interacting with MQTT Broker using CLI

****Task 2**** Check if the mosquitto service is listening.

- By default, mosquitto service will be running on ip 127.0.0.1 and port 1883
- For windows you can check using below command from a Terminal window

```
netstat -an | findstr 1883
```

```
C:\Users\rajes>netstat -an | findstr 1883
TCP    127.0.0.1:1883      0.0.0.0:0          LISTENING
TCP    [::]:1883          [::]:0             LISTENING
```

- For Linux use below command.. and you should see the response as follows

```
netstat -ltn
```

```
Active Internet connections (only servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	0.0.0.0:1883	0.0.0.0:*	LISTEN
tcp6	0	0	:::1883	:::*	LISTEN

- For Mac below is command and response

```
lsof -nP -iTCP:1883 | grep LISTEN
```

```
mosquitto ... user 4u IPv4 ... 0t0 TCP 127.0.0.1:1883 (LISTEN)
```

```
mosquitto ... user 5u IPv6 ... 0t0 TCP [::]:1883 (LISTEN)
```

Interacting with MQTT Broker using CLI

****Task 3**** Publish some message to the local broker with the topic **testing/lab1**

- **Publish** - Use below command-on-command line to publish a message

```
mosquitto_pub -t "topic_name" -m "message"
```

- Messages **message** will be published to a topic **topic_name**
- By default, mosquitto_pub command publishes to the localhost and the port 1883.

We can enforce another host or port in this command as below -

```
mosquitto_pub -h "localhost" -p port# -t "topic_name" -m "message"
```

- For eg -

```
mosquitto_pub -h "1.2.3.4" -p 5000 -t "topic_name" -m "message"
```

```
mosquitto_pub -h "mqtt.gl.in" -p 5000 -t "topic_name" -m "message"
```

Interacting with MQTT Broker using CLI

****Task 4**** Subscribe message to the local broker with the topic **testing/lab1**

- **Subscribe** - Use below command-on-command line to publish a message

```
mosquitto_sub -t "topic_name"
```

- By default localhost and the port 1883 is subscribed

We can enforce another host or port in this command as below -

```
mosquitto_sub -h "localhost" -p 1883 -t "topic_name"
```

- Open another Terminal and execute an instruction to subscribe to the topic testing/lab1

```
mosquitto_sub -t "testing/lab1"
```

- Go back to the Terminal 1 and execute the publish instruction and you should now see the message printed on the Terminal 2

Lab 3 - MQTT Topic – Deep Dive

MQTT Topics

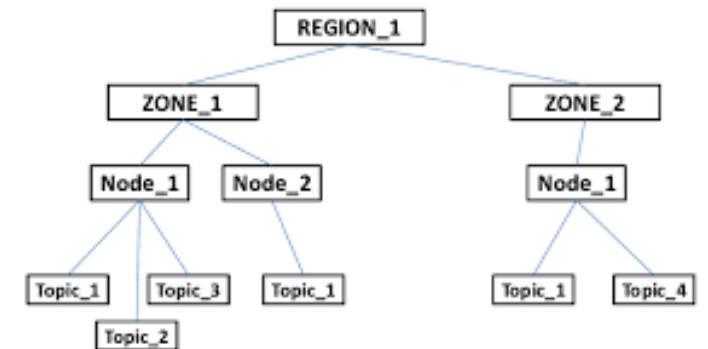
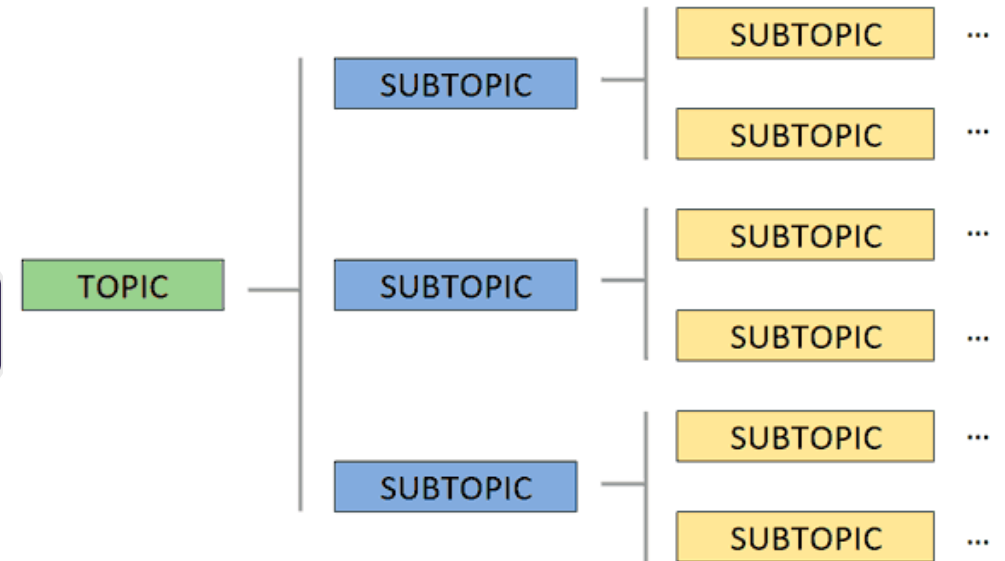
- UTF-8 strings used by the broker to filter and route messages to clients.
- Topics are hierarchical and separated by /.
- Topics are case-sensitive and must contain at least one character.
- Best practice: avoid spaces and keep topic names short and consistent.
- Examples - Devices/building1/floor3/room8/temperature
 - level0/level1/level2/level3

MQTT Wildcards : used in subscriptions, not in publish topic names.

- + = single-level wildcard, used for one topic level.
Example: devices/building1/+/room8/temperature
- # = multi-level wildcard, used at the end of a topic filter.
Example: devices/building1/#
- Avoid subscribing to # in large systems because it can generate too much traffic.

Reserved System Topics

- Topics starting with \$ are reserved for broker and not matched by #.
 - \$SYS/broker/clients/connected
 - \$SYS/broker/clients/disconnected
 - \$SYS/broker/clients/total
 - \$SYS/broker/messages/sent
 - \$SYS/broker/uptime



Lab 3 - MQTT Topic – Deep Dive

****Task 5:**** Subscribe to a topic B1/+ /R1 in the Terminal 1

```
mosquitto_sub -t "B1/+ /R1"
```

In the Terminal 2, publish to different topics and look at the values showing up in Terminal 1

```
mosquitto_pub -t "B1/F1/R1" -m "Message to B1/F1/R1"
```

```
mosquitto_pub -t "B1/F2/R1" -m "Message to B1/F2/R1"
```

Lab 3 - MQTT Topic – Deep Dive

****Task 6:**** Subscribe to a topic devices/# in the Terminal 1

```
mosquitto_sub -t "B1/#"
```

In the Terminal 2, publish to different topics and look at the values showing up in Terminal 1

```
mosquitto_pub -t "B1/F1" -m "Message to B1/F1"
```

```
mosquitto_pub -t "B1/F2" -m "Message to B1/F2"
```

Lab 4 – Other Attributes

- **Publishing the contents of a file as the message**

- To publish the contents of a file as the message, use the flag -f instead of -m as follows:
`mosquitto_pub -h "localhost" -p 1883 -t "topic_name" -f file_path`

- ****Task 7:****

- Create a file named message.json, insert the following contents to it and save the file

```
{
  "device_name": "GL248",
  "device_type": "counter"
  "timestamp": "2021-01-14 13:13:13"
  "value": 45
}
```
- Open a terminal and navigate to the directory where you saved the above file and publish the contents of this file to the topic file publish

```
mosquitto_pub -t "filepublish" -f message.json
```

Lab 4 – Other Attributes

- **Debugging the messages**

- One of the simplest ways of debugging the communication between the client and broker is by adding the flag -d while you publish or subscribe to the broker. This allows you to look at various packets exchanged between the client and the broker.

- ****Task 8:**** Publishing with debug enabled Command

```
mosquitto_pub -d -t "topic_name" -m "{ 'keys': 'values' }"
```

Response

```
1609685374: New connection from ::1:59928 on port 1883.
```

```
Client (null) sending CONNECT
```

```
1609685374: New client connected from ::1:59928 as auto-A49A14D9-6030-B438-2599-42C62077CF16 (p2, c1, k60).
```

```
Client (null) received CONNACK (0)
```

```
Client (null) sending PUBLISH (d0, q0, r0, m1, 'topic_name', ... (17 bytes))
```

```
Client (null) sending DISCONNECT
```

```
1609685374: Client auto-A49A14D9-6030-B438-2599-42C62077CF16 disconnected.
```

Lab 4 – Other Attributes

- ****Task 9:**** Subscribing with debug enabled

- Command

```
mosquitto_sub -d -t "topic_name"
Response
Client (null) sending CONNECT
Client (null) received CONNACK (0)
Client (null) sending SUBSCRIBE (Mid: 1, Topic: topic_name, QoS: 0, Options: 0x00)
Client (null) received SUBACK
Subscribed (mid: 1): 0
#And every 60 seconds, you can find the following messages printed
Client (null) sending PINGREQ
Client (null) received PINGRESP
```

The reason why you see the term null in the above responses is because the client wasn't assigned with an ID

Lab 5 – Assigning Client Id

- **Assigning Client ID** Mosquitto clients can be configured with a client ID (preferably unique) with the -i flag. Having a client id is very helpful while debugging and it is a recommended practice.
- ****Task 10:**** Let's repeat the Task 8 with addition of the -i flag and observe the result

```
mosquitto_sub -d -i "terminal_subscriber" -t "topic_name"
```

Response

```
Client terminal_subscriber sending CONNECT
```

```
Client terminal_subscriber received CONNACK (0)
```

```
Client terminal_subscriber sending SUBSCRIBE (Mid: 1, Topic: topic_name, QoS: 0, Options: 0x00)
```

```
Client terminal_subscriber received SUBACK
```

```
Subscribed (mid: 1): 0
```

Quick recap

Task 1	Setting up mosquito
Task 2	Check if mosquito is listing
Task 3	Publish message to topic
Task 4	Subscribe message from topic
Task 5 & 6	Topic deep dive – using wildcard + & #
Task 7	Publishing file in place of message
Task 8 & 9	Debugging message - publisher and subscriber
Task 10	Assigning Client Id

Next – Configuring Listener

Lab 6 – Configuring Listener

- Configurations for the mosquitto broker is defined in the mosquitto.conf, usually located in the following path

```
Windows - C:\Program Files\mosquitto\  
Linux - /etc/mosquitto/  
Mac - /usr/local/etc/mosquitto/
```

- Initially, the mosquitto.conf file contains all the configurations in commented state. Create a backup of this and save it with another name, say mosquitto.conf.bak for future reference.
- There are a number of things that can be configured in this configuration file such as:
 - the listener - bind port and bind interface
 - message size limit
 - max queued message
 - maximum number of connections
 - location of password file
 - SSL/TLS support
 - ... (refer the original mosquitto.conf file)
- In this section, we will only configure the listener. Following is the format of configuring a listener

```
Conf  
listener [bind_port] [bind_interface]
```

Lab 6 – Configuring Listener

Port binding

****Task 11**** Edit the mosquitto.conf file, empty it out and add the following line only

```
listener 5000
allow_anonymous true
```

and now restart the mosquitto broker

Windows Go to Task Manager → Services → mosquitto, right click and select Restart

Linux `sudo systemctl restart mosquitto.service`

Mac

#Stop the broker

```
kill all mosquito
```

#Start the broker with specified configuration file in the background

```
/usr/local/sbin/mosquitto -c /usr/local/etc/mosquitto/mosquitto.conf -d
```

Now the broker will have started in the port 5000. Any further publish/subscribe you make to the broker explicitly needs to mention the port with the -p 5000 flag

Lab 6 – Configuring Listener

****Task 12**** : **Interface binding** : Publish and subscribe messages with the newly configured port

- *#Publish example* `mosquitto_pub -p 5000 -t "topic_name" -m "message"`
- *#Subscribe example* `mosquitto_sub -p 5000 -t "topic_name"`

This was an example of how to configure a different port where the mosquitto broker listens. Similarly, we can restrict the network in which the broker will be available by binding it to one or more of your computer's network interfaces

- To restrict the broker to function only for the localhost, the listener can be configured as

```
listener 1883 localhost
#or listener 1883 127.0.0.1
allow_anonymous true
```
- If your computer is connected to a LAN and let's say your computer has the IP address 192.168.1.105, then you could use this as the `_bindinterface` to restrict the usage of your broker only to your LAN

```
listener 1883 192.168.1.105
allow_anonymous true
```
- To open up your broker for all the network interfaces, you could use the `bind_address` as 0.0.0.0

```
listener 1883 0.0.0.0
```
- This is the default configuration even if the `bind_port` and `bind_interface` is not mentioned explicitly

```
allow_anonymous true
```

Lab 6 – Configuring Listener

****Task 13****

- Bind the broker connection to accept requests only from localhost, restart the broker and try publishing messages to the broker from another device in your computer's LAN
 - You can use an MQTT application in your smartphone to publish messages to the broker. Make sure your smartphone and the computer are on the same LAN
 - Since the broker is configured to accept connections only from the localhost, other devices in your LAN should not be able to connect, publish or subscribe to the broker.
 - Now edit the configuration file and bind the computer's LAN IP address, restart the broker and verify if other devices on the LAN are able to connect, publish and subscribe to the broker.